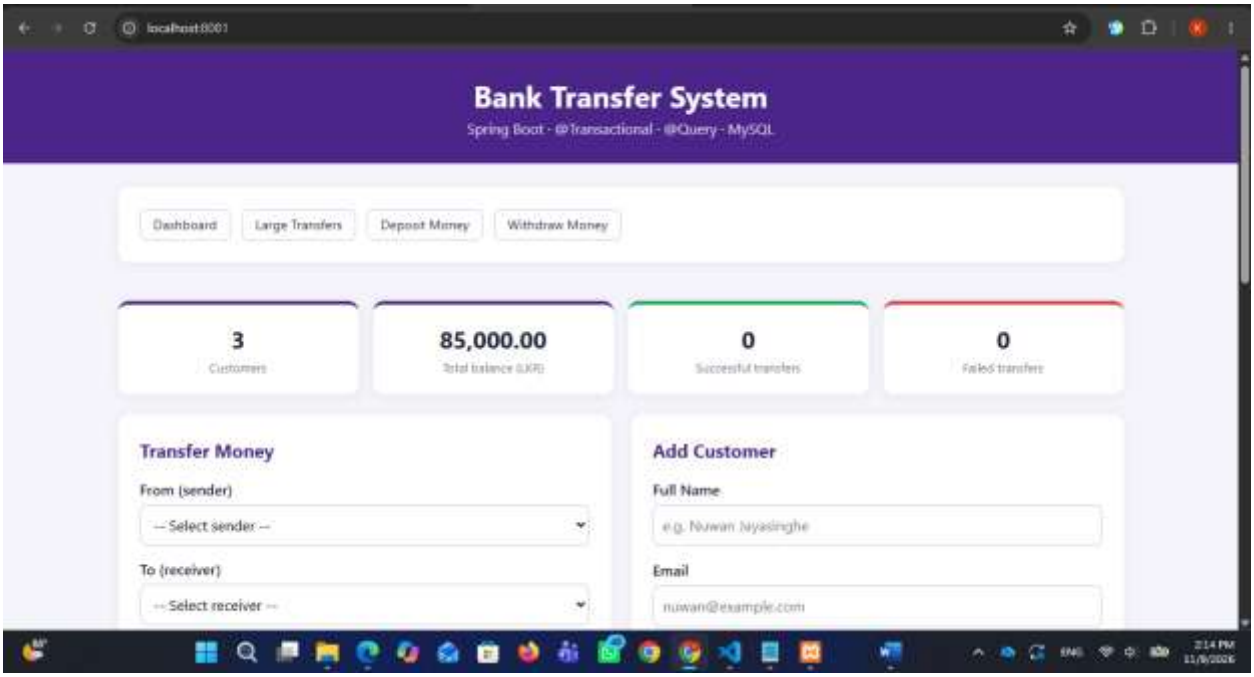
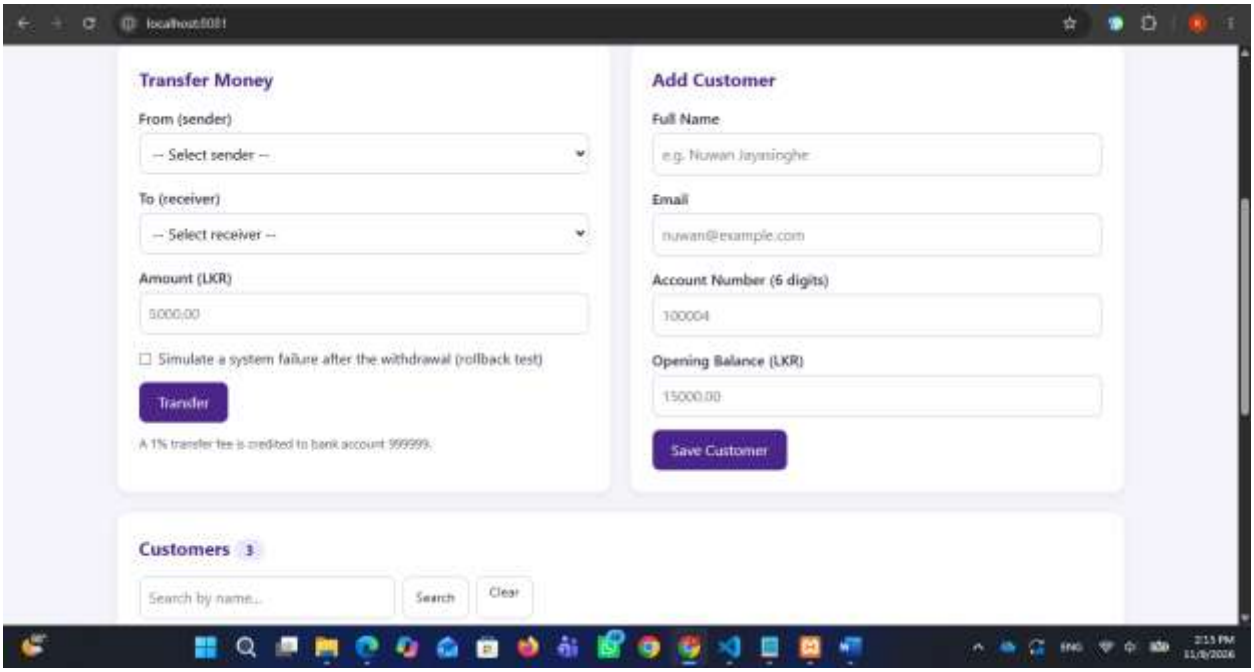




1. Project Overview & Introduction Section

- **Objective:** Build a secure Full-Stack Bank Transfer System using Spring Boot, Thymeleaf, Spring Data JPA, and MySQL to demonstrate robust transaction management (`@Transactional`), custom repository queries (`@Query`), and data integrity features.
- **Tech Stack:** Java 21, Spring Boot 4.0.9, MySQL 8.0, Thymeleaf, Maven.

- **Application Execution Instructions:**
    1. Start the MySQL database server (e.g., via XAMPP Control Panel).
    2. Open a terminal in the project root folder (`bank-transfer-app`) and run the command:

       Bash

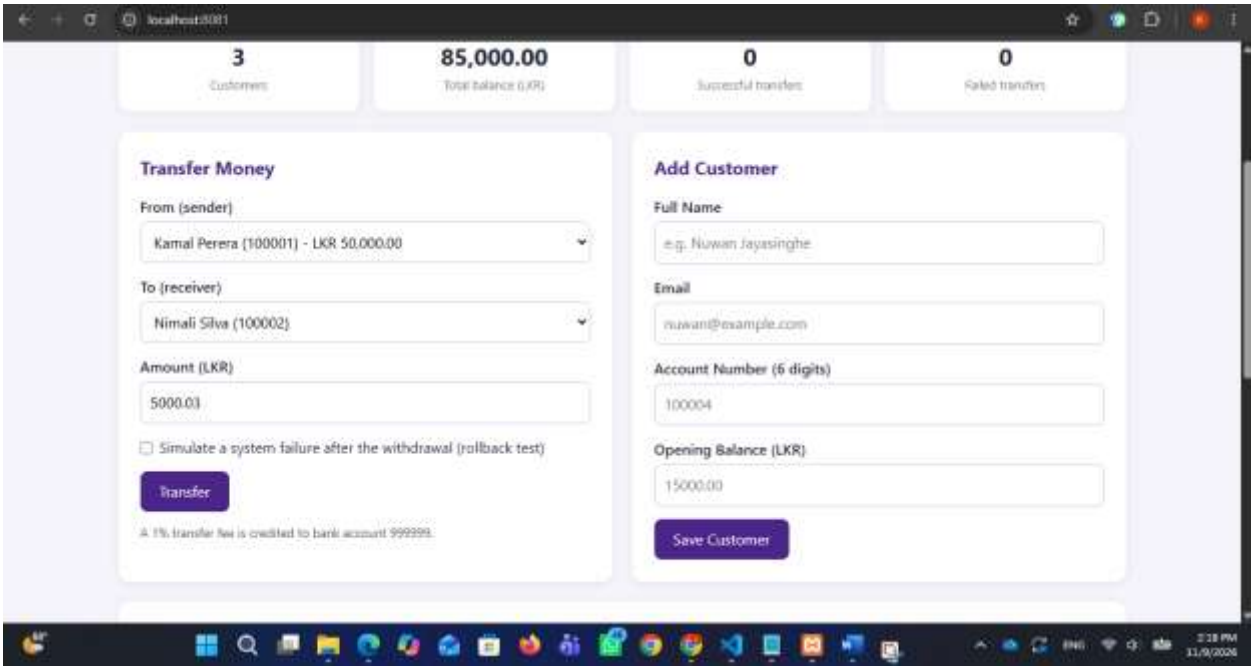
       ```
       mvnw.cmd spring-boot:run
       ```

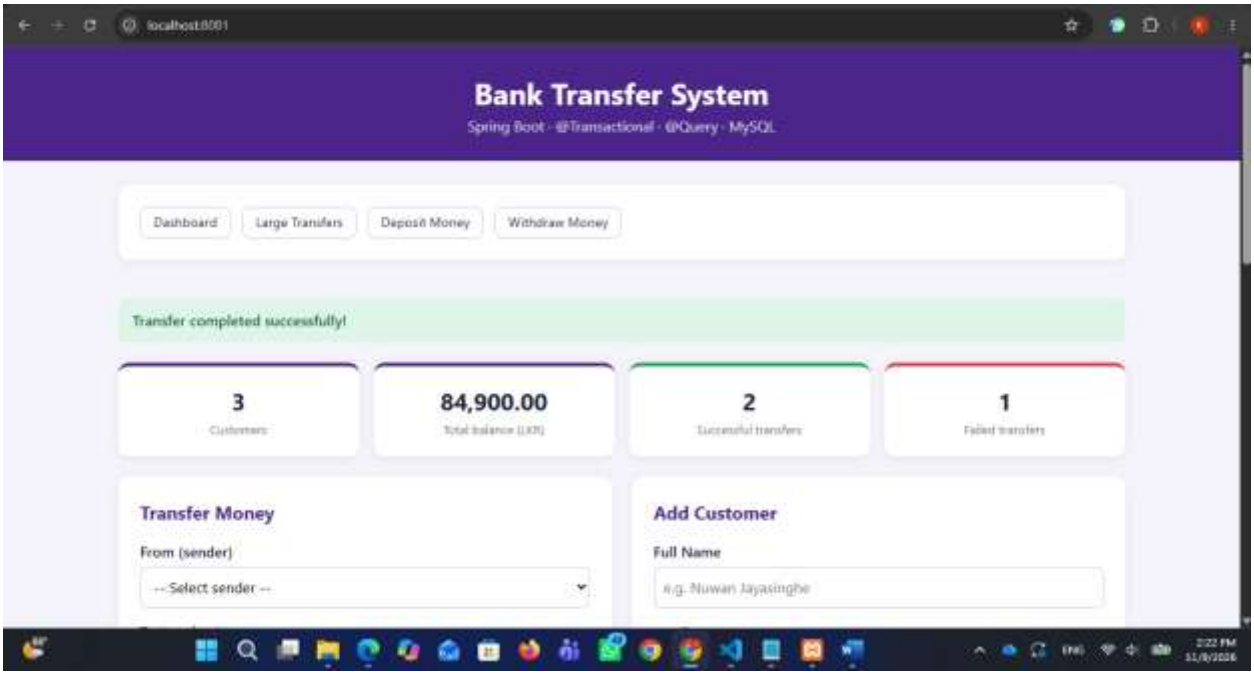
    3. Open a web browser and access the local dashboard via URL: `http://localhost:8081`.

Test Cases & Screenshots Mapping

Test Case 1 (Successful Money Transfer):

Verify that transferring funds between valid customer accounts (e.g., from Kamal Perera to Nimali Silva) updates balances correctly and records a `SUCCESS` status entry in the transaction history table.

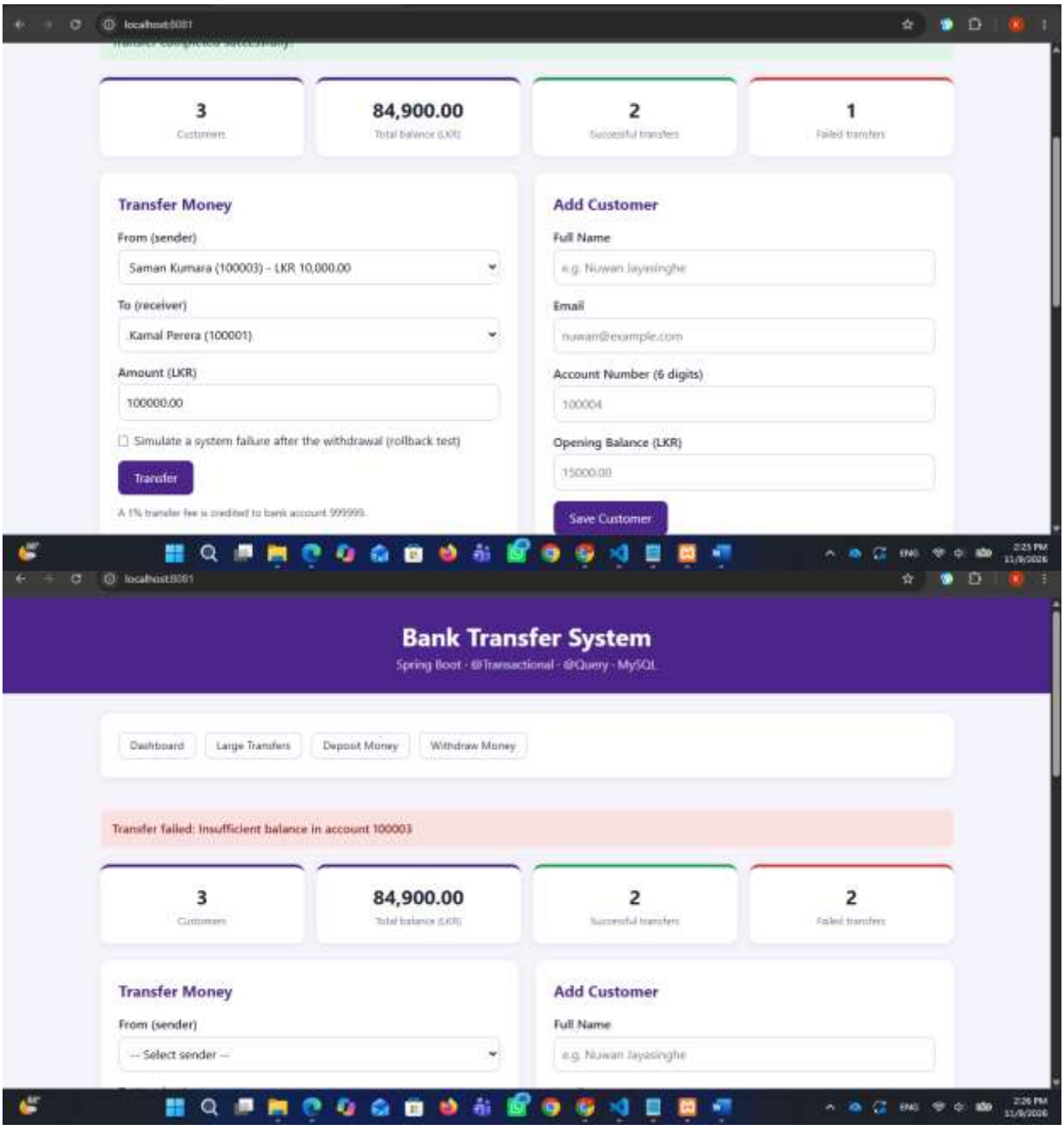




"`Transfer completed successfully!`" alert message, updated customer totals, and the successful transaction record here.

Test Case 2 (Insufficient Balance Error Test):

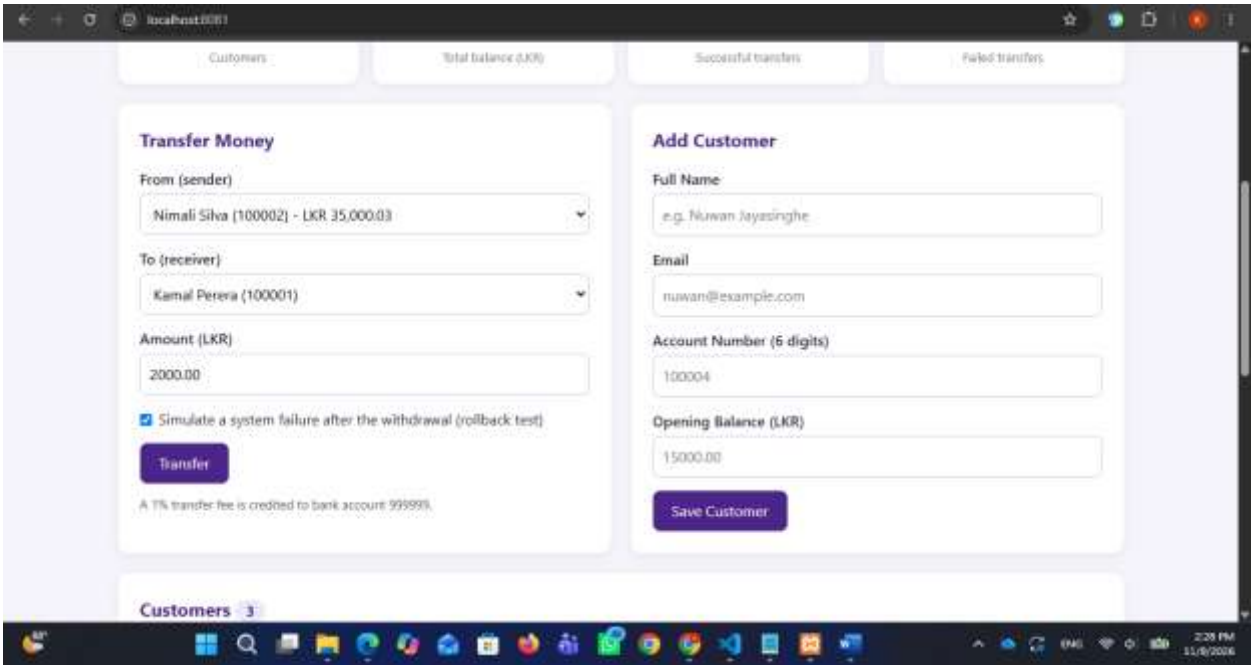
Verify that attempting a transfer with an amount exceeding the sender's available balance triggers a validation failure and records a `FAILED` transaction.

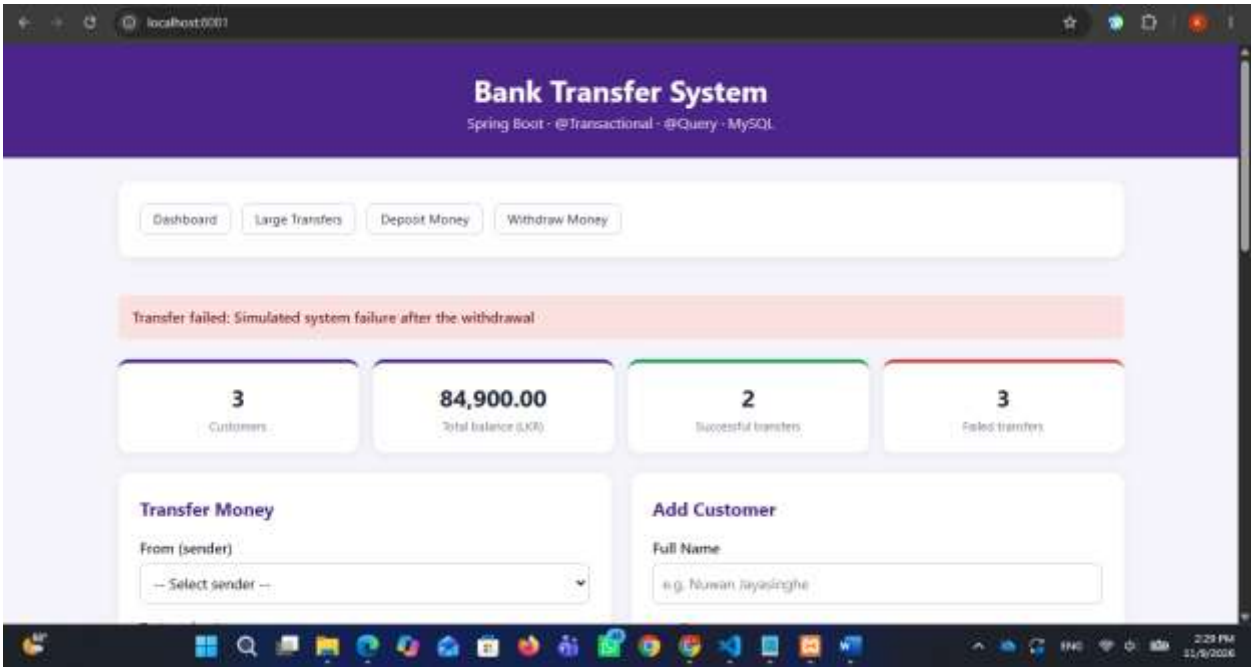


insufficient balance (e.g., trying to transfer LKR 100,000 from Saman Kumara) here.

Test Case 3 (Rollback Test / Simulate System Failure):

Verify the ACID properties and transaction atomicity by simulating a system crash immediately after the withdrawal step. Ensure that the debited funds are fully rolled back and no money is lost, while logging a `FAILED` record using independent propagation.



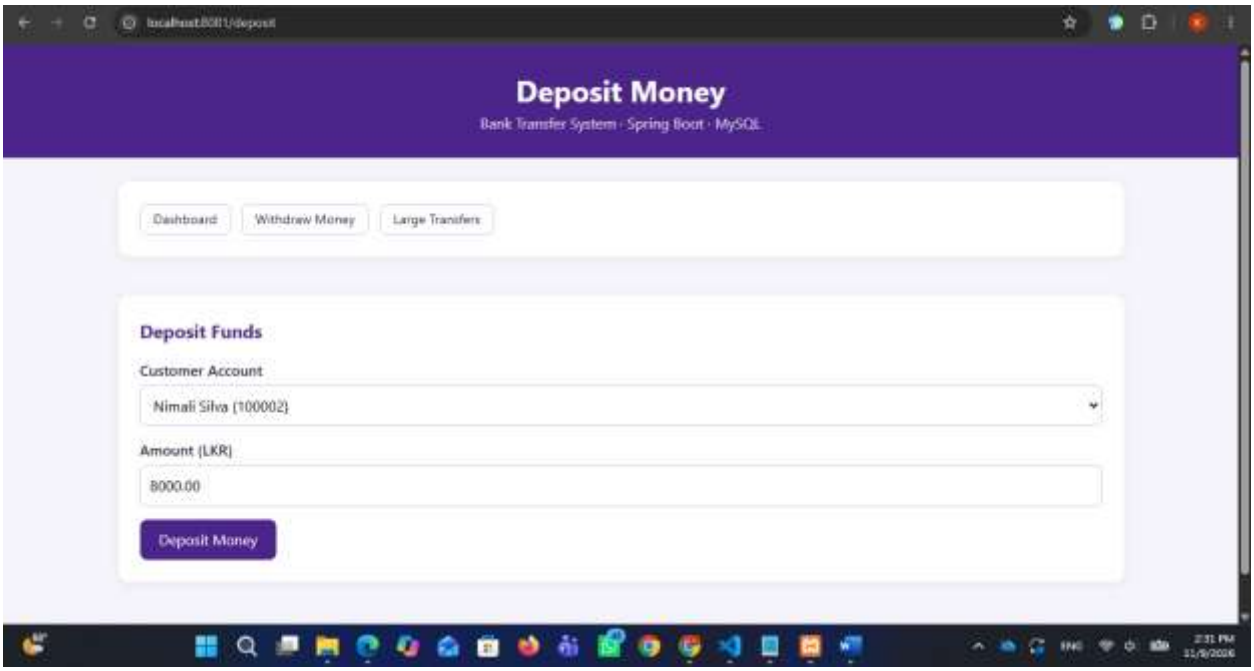


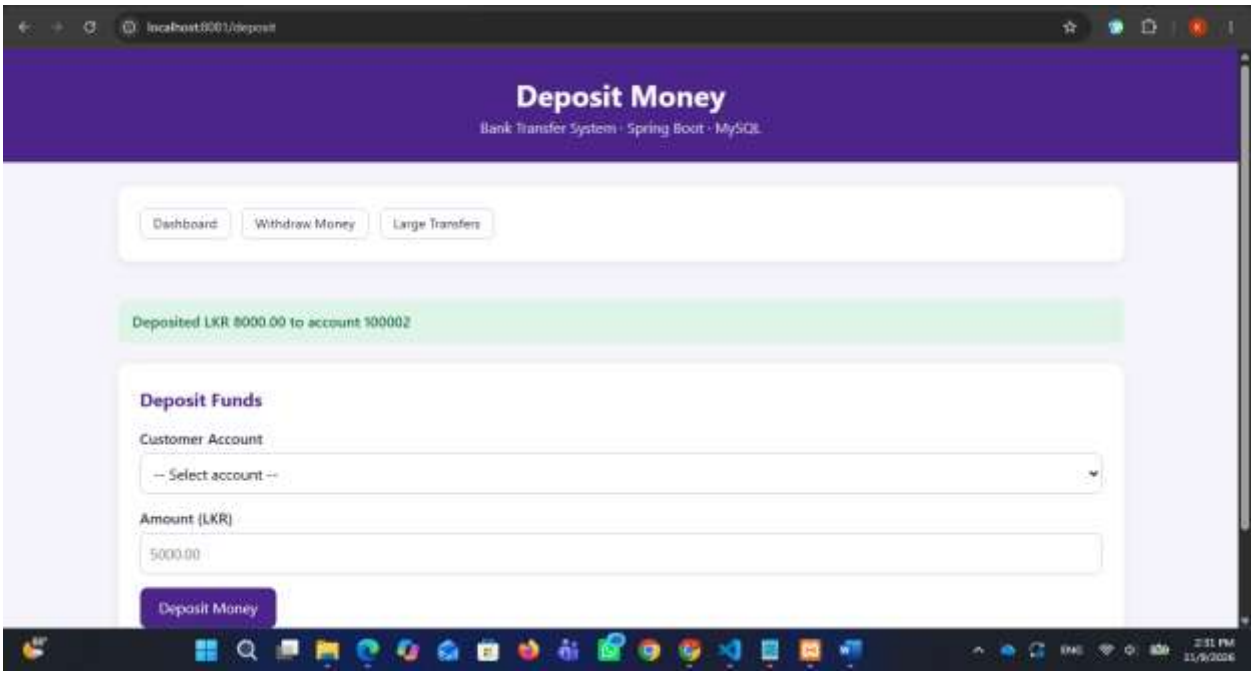
"`Transfer failed: Simulated system failure after the withdrawal`" warning banner and the resulting `FAILED` status entry here.

Test Case 4 (Extension Features)

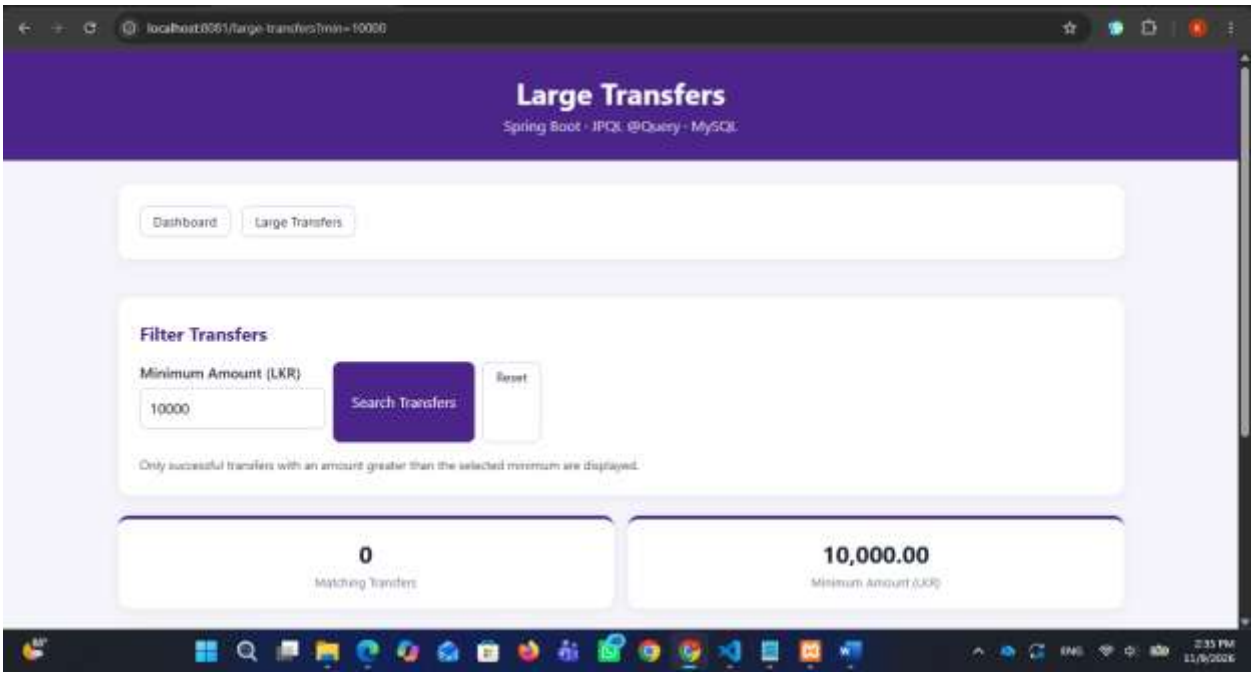
Test advanced extension features including direct cash deposits, filtering large transfers above a threshold using custom JPQL queries, and inspecting individual account audit histories.

Deposit



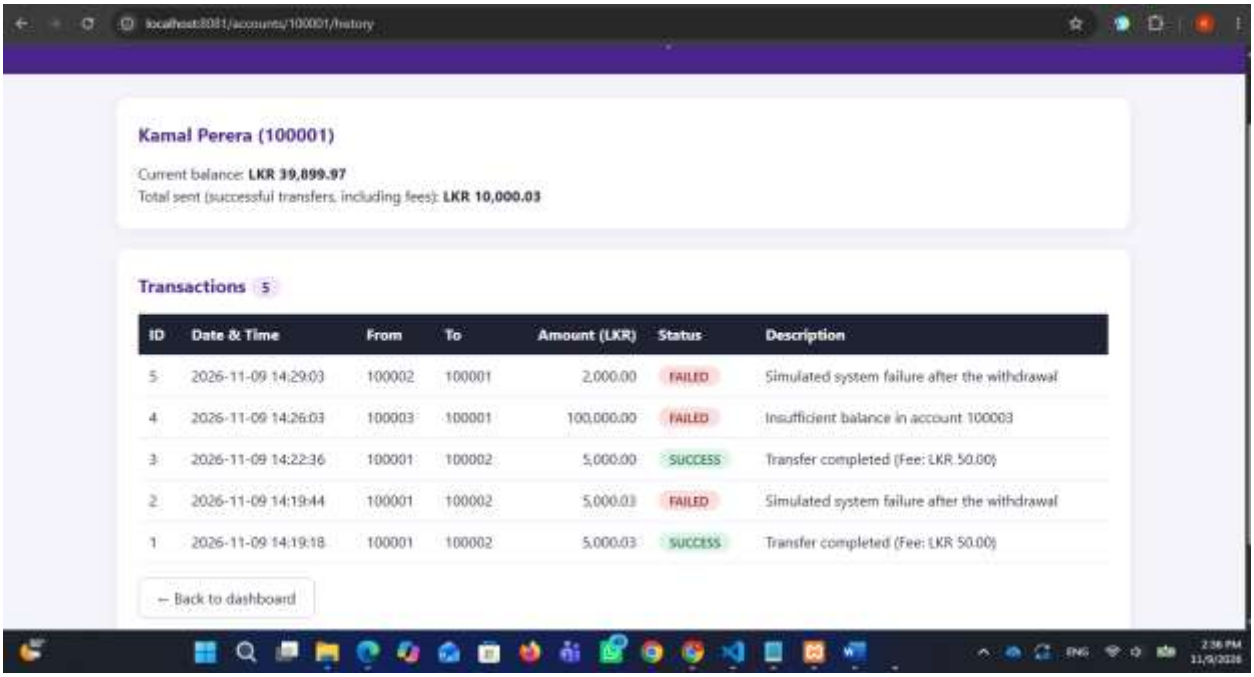


Large Transfers

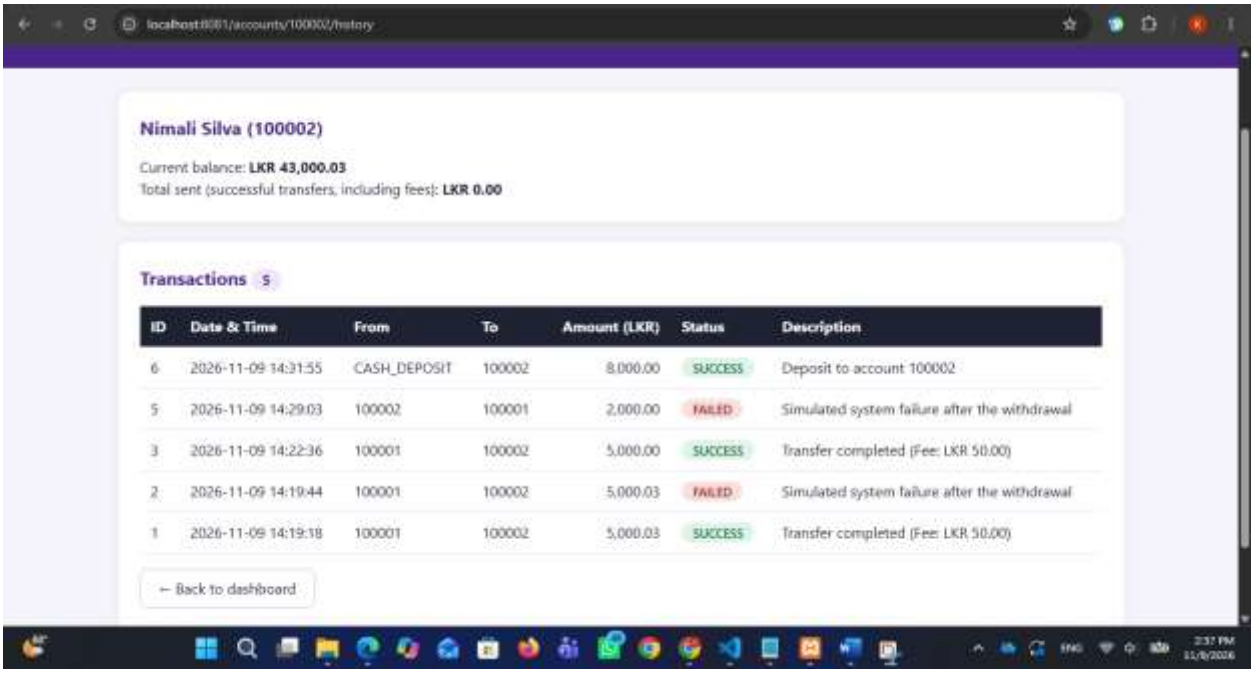


History

History Of Kamal Perera



History Of Nilmini Silva



Nimali Silva (100002)

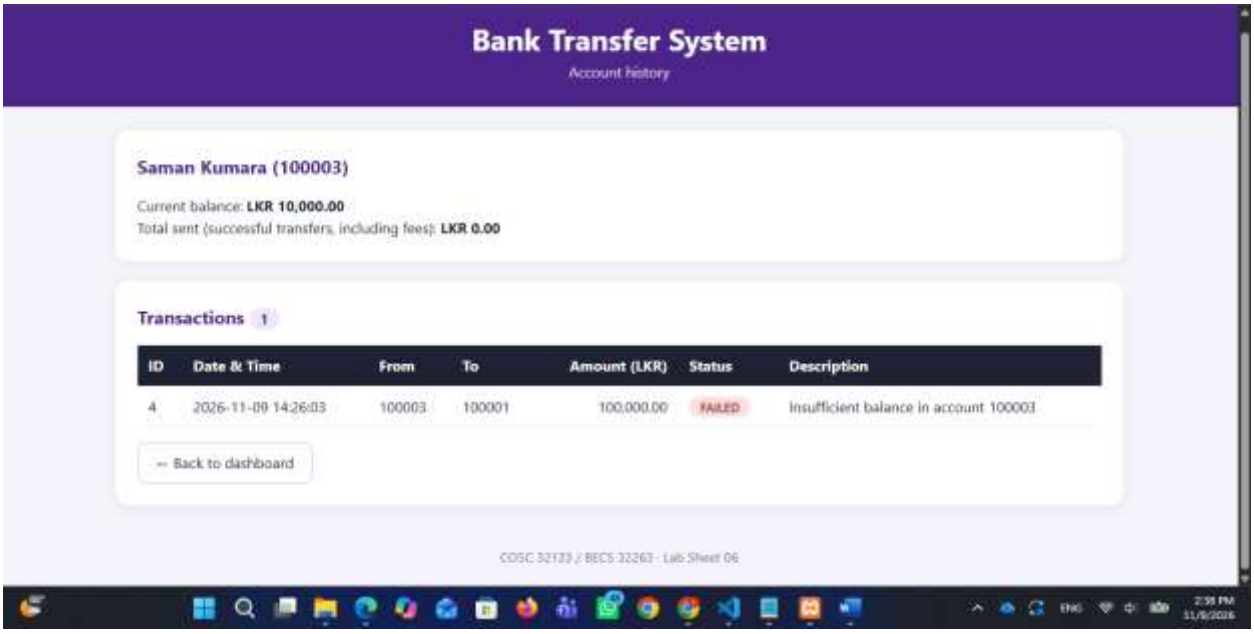
Current balance: **LKR 43,000.03**
Total sent (successful transfers, including fees): **LKR 0.00**

Transactions 5

ID	Date & Time	From	To	Amount (LKR)	Status	Description
6	2026-11-09 14:31:55	CASH_DEPOSIT	100002	8,000.00	SUCCESS	Deposit to account 100002
5	2026-11-09 14:29:03	100002	100001	2,000.00	FAILED	Simulated system failure after the withdrawal
3	2026-11-09 14:22:36	100001	100002	5,000.00	SUCCESS	Transfer completed (Fee: LKR 50.00)
2	2026-11-09 14:19:44	100001	100002	5,000.03	FAILED	Simulated system failure after the withdrawal
1	2026-11-09 14:19:18	100001	100002	5,000.03	SUCCESS	Transfer completed (Fee: LKR 50.00)

← Back to dashboard

History Of Saman Kumara



Bank Transfer System

Account history

Saman Kumara (100003)

Current balance: **LKR 10,000.00**
Total sent (successful transfers, including fees): **LKR 0.00**

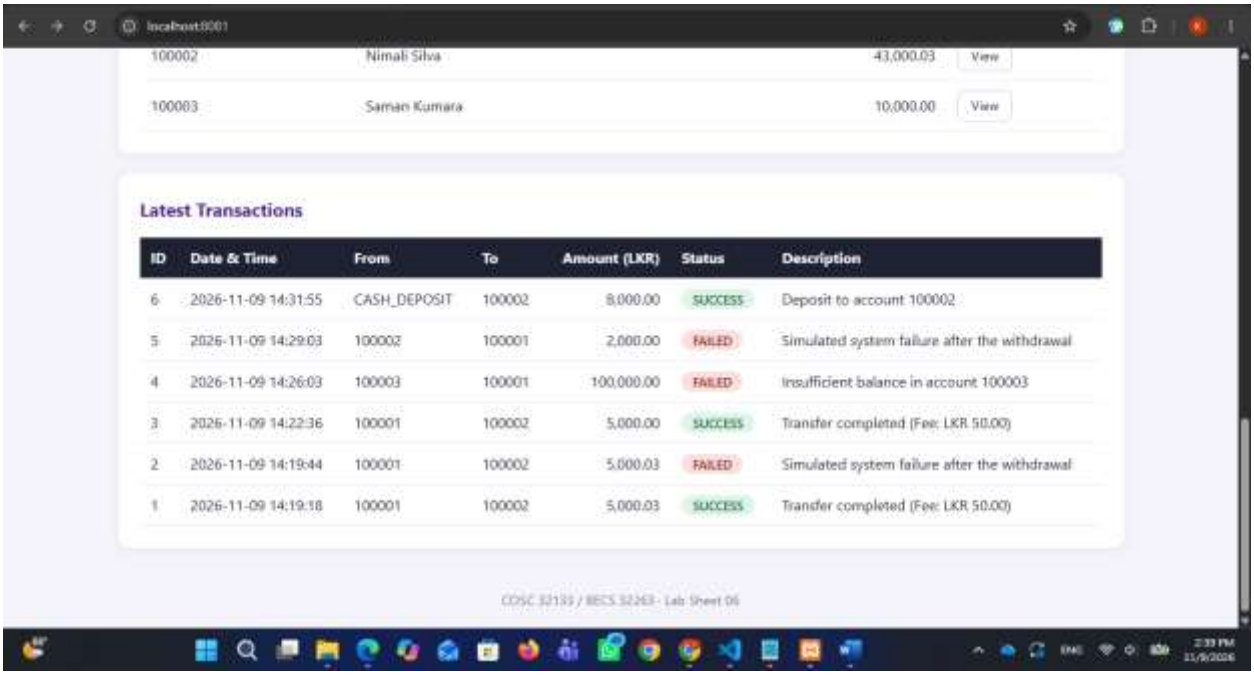
Transactions 1

ID	Date & Time	From	To	Amount (LKR)	Status	Description
4	2026-11-09 14:26:03	100003	100001	100,000.00	FAILED	Insufficient balance in account 100003

← Back to dashboard

COSC 32133 / BECS 32263 - Lab Sheet 06

Latest Transactions



(a) What would have happened to the money in test 4 without

Without `@Transactional`, database operations execute independently as auto-committed steps. The withdrawal (`debit`) from the sender would commit instantly before the simulated exception occurs. When the exception triggers, the receiver's deposit (`credit`) would fail to execute, causing money to permanently vanish from the system without returning to the sender.

(b) Why is `recordFailure` in a separate class with `REQUIRES_NEW`?

Spring's transaction management relies on proxy-based interception, meaning self-invocation within the same class bypasses transaction boundaries. Moving failure logging into a separate service class (`TransactionAuditService`) with `propagation = Propagation.REQUIRES_NEW` enables Spring to suspend the main failing transaction (which rolls back) and open an independent, separate transaction to successfully save the `FAILED` audit record.

(c) Why does `debit` use `UPDATE ... WHERE balance >= :amount` instead of reading the balance first?

Reading the balance via a separate query prior to updating introduces race conditions and lost updates in multi-user concurrent environments. The atomic JPQL/SQL `UPDATE` statement evaluates the condition (`balance >= :amount`) and subtracts funds in a single database-level lock operation, ensuring thread-safe balance validation without requiring expensive pessimistic locking.